

Learnify - Online Learning Platform

Integrated Project Report

Submitted in Partial Fulfilment of the requirement for the Course Integrated Project (22CS038)
of

Computer Science and Engineering

BE Batch - 2023

In

May 2026

Under the Guidance of:

Dr. Gurtej Kaur
Associate Professor

Submitted By:

Aadeesh Jain, 2310991501 (G-17)
Aakash Vij, 2310991502 (G-17)
Anmol, 2310991525 (G-17)
Garv Gupta, 2310991573 (G-17)

Department of Computer Science and Engineering

Chitkara University Institute of Engineering & Technology,
Chitkara University, Punjab

Certificate

This is to certify that the project entitled "Learnify - Online Learning Platform" has been submitted for the Bachelor of Computer Science Engineering at Chitkara University, Punjab during the academic semester January 2026 to May 2026. The report has been prepared by Aadeesh Jain, Aakash Vij, Anmol, and Garv Gupta under my guidance and supervision.

The submitted work demonstrates the design and implementation of a web-based learning platform developed using Node.js, Express.js, EJS templates, MongoDB, Mongoose, and supporting middleware for security, routing, validation, and deployment.

Dr. Gurtej Kaur

Associate Professor - CSE

Candidate's Declaration

We, Aadeesh Jain (2310991501), Aakash Vij (2310991502), Anmol (2310991525), and Garv Gupta (2310991573), B.E. - 2023 of Chitkara University, Punjab, hereby declare that the Integrated Project Report entitled "Learnify - Online Learning Platform" is our original work carried out as part of the Integrated Project course.

The report has not been submitted to any other university or institution for the award of any degree or diploma. All sources, tools, technologies, and references used during the development of the project have been acknowledged appropriately.

Aadeesh Jain	Aakash Vij	Anmol	Garv Gupta
2310991501	2310991502	2310991525	2310991573

ACKNOWLEDGEMENT

It is our pleasure to express sincere gratitude to everyone who contributed directly or indirectly to the successful completion of this project. We are thankful to our guide, Dr. Gurtej Kaur, for continuous support, valuable suggestions, and guidance throughout the development of Learnify.

We also thank the Department of Computer Science and Engineering, Chitkara University, for providing the academic environment, technical exposure, and resources required to complete this project. Finally, we extend appreciation to our teammates for their coordination, research, implementation effort, and testing support.

ABSTRACT

Learnify is a web-based online learning platform developed to provide students with an organized and accessible environment for browsing courses, viewing playlists, registering accounts, exploring teachers, and purchasing courses. The project focuses on creating a simple learning workflow where users can authenticate, browse educational content, and complete course purchase details through a structured interface.

The backend of the application is developed using Node.js and Express.js. Express routes manage page rendering, authentication, playlist operations, and course-purchase flows. EJS is used as the server-side view engine to render dynamic pages, while static assets such as CSS, images, thumbnails, and course media are served from the public directory.

MongoDB is used as the database with Mongoose models for users, videos, and purchases. User passwords are protected with bcrypt hashing through a Mongoose pre-save hook, improving security for stored credentials. The application also uses middleware such as Helmet, CORS, Morgan, compression, body-parser, and cookie-parser to support security, request logging, performance, and request parsing.

Learnify is designed for deployment on Render with environment-based configuration. The project demonstrates practical backend development, database connectivity, route organization, authentication, templating, and deployment practices in a full-stack web application.

Table of Contents

S.No.	Content	Page No.
1.	Abstract and Keywords	
2.	Introduction to the Project 2.1 Background 2.2 Problem Statement	
3.	Software and Hardware Requirement Specification 3.1 Methods 3.2 Programming Environment 3.3 Requirements to Run the Application	
4.	High Level and Low Level Design	
5.	Database Analysis, Design and Implementation	
6.	Program Structure and Database Connections	
7.	Code Implementation and GUI Construction	
8.	System Testing	
9.	Conclusion	
10.	Future Scope	
11.	Bibliography and References	

Abstract and Keywords

Learnify is a full-stack online learning web application that provides core learning-platform features such as user registration, login, course listing, playlist viewing, teacher pages, course details, and payment-record storage. The system is implemented using Node.js, Express.js, EJS, MongoDB, and Mongoose, with static assets and templates organized into separate folders for maintainable development.

The project uses Express routing to separate page routes, authentication routes, playlist routes, and purchase routes. MongoDB stores persistent application data, while Mongoose schemas define the structure of users, videos, and purchases. The platform includes course data for HTML, CSS, JavaScript, Java, and C++ with tutors, prices, thumbnails, levels, descriptions, and video lists.

KEYWORDS

- Online Learning Platform
- Node.js
- Express.js
- EJS
- MongoDB
- Mongoose
- bcrypt
- Authentication
- Course Purchase
- Playlist Management
- Render Deployment
- Full Stack Development

2. Introduction to the Project

The rapid growth of digital education has increased the need for platforms that can organize learning material, present courses clearly, and support students through a simple web interface. Learnify is designed as an online learning system that combines course discovery, playlist-based video learning, authentication, teacher information, and purchase handling in one application.

The application follows a server-rendered full-stack architecture. Node.js provides the runtime environment, Express.js manages routing and middleware, EJS renders dynamic pages, and MongoDB stores user, playlist, and purchase data. The project is structured so that routes, models, views, public assets, and configuration files are separated clearly.

2.1 Background

Traditional learning resources are often scattered across multiple websites, files, and video platforms. Students may face difficulty finding structured course content, understanding the tutor profile, and accessing playlists in a single location. Learnify addresses this problem by presenting course content through a centralized learning interface.

The project uses modern JavaScript backend development practices. Server-side rendering with EJS keeps the interface lightweight, while MongoDB allows flexible storage for users, videos, and purchase records. Middleware improves logging, security headers, response compression, request parsing, and cross-origin handling.

2.2 Problem Statement

The problem addressed by Learnify is the lack of a simple and organized course platform for students to register, browse courses, access playlists, learn from tutor information, and record course purchases. The system must be easy to run locally, configurable for deployment, secure for basic authentication, and maintainable for future expansion.

3. Software and Hardware Requirement Specification

3.1 Software Requirements

Category	Requirement
Runtime	Node.js version 18 or above
Backend Framework	Express.js
Template Engine	EJS
Database	MongoDB local server or MongoDB Atlas
ODM	Mongoose
Authentication Security	bcryptjs password hashing
Middleware	helmet, cors, morgan, compression, body-parser, cookie-parser
Deployment	Render web service with environment variables
Development Tools	Visual Studio Code, browser, terminal, Git/GitHub

3.2 Hardware Requirements

Requirement Type	Minimum	Recommended
Processor	Dual-core processor	Quad-core processor or better
RAM	4 GB	8 GB or more
Storage	1 GB free space	5 GB free space for project, assets, and dependencies
Network	Internet for package install/deployment	Stable broadband connection

3.3 Methods

- Requirement analysis was performed by identifying the student, course, playlist, teacher, and purchase flows.
- Backend development was completed using modular Express routers and Mongoose models.
- Database design was implemented through schemas for users, videos, and purchases.
- Interface construction was completed using EJS views and static CSS/images.
- Testing was carried out by running the server, checking routes, validating form input, and confirming database operations.

3.4 Programming Environment

The programming environment includes Node.js, npm, Express.js, EJS, MongoDB, and Mongoose. The application can be started with npm start after dependencies are installed and the MongoDB connection string is configured.

3.5 Requirements to Run the Application

```
npm install
copy .env.example .env
npm run seed
npm start
```

The default local URL is `http://localhost:8080`. The `.env` file stores the `PORT` and `MONGODB_URI` values required by the server.

4. High Level and Low Level Design

4.1 High Level Design

The high-level design of Learnify consists of four main layers: presentation layer, routing layer, business/data access layer, and database layer. The presentation layer contains EJS templates and public assets. The routing layer maps user requests to controllers implemented through Express routers. The data layer uses Mongoose models, and MongoDB stores persistent records.

Layer	Main Files	Responsibility
Presentation Layer	views/*.ejs, public/css, public/images	Displays pages such as login, register, home, courses, playlists, teacher pages, and payment pages.
Routing Layer	routes/pages.js, routes/auth.js, routes/playlist.js, routes/buyCourse.js	Handles page navigation, authentication, playlist data, and course-purchase requests.
Model Layer	models/User.js, models/Video.js, models/Purchase.js	Defines database schemas and helper methods.
Database Layer	db.js, MongoDB	Connects to MongoDB and stores users, videos, and purchase records.

4.2 Low Level Design

At the low level, each feature is separated into route handlers and supporting models. The authentication module validates email and password fields, checks existing users, hashes passwords through the User schema, and compares passwords during login. The playlist module fetches videos from MongoDB and renders playlist pages. The purchase module validates payment form fields and stores purchase records.

Module	Input	Processing	Output
Registration	Email, password	Validate fields, check duplicate email, hash password, save user	Login page with success message
Login	Email, password	Find user, compare password hash	Home page or error message
Playlist	GET request or video form data	Fetch or insert videos through Video model	Playlist page or save response
Course Purchase	Student name, email, phone, payment method	Validate form and create Purchase document	Payment success page

5. Database Analysis, Design and Implementation

MongoDB is used as the database for Learnify. It is suitable for this project because the data model is flexible and can store documents for users, videos, purchases, and course-related records. Mongoose is used to define schemas, validation rules, and model methods.

5.1 Main Collections

Collection	Fields	Purpose
users	email, password	Stores registered user accounts with hashed passwords.
videos	title, videoUrl, thumbnail	Stores video playlist content displayed on playlist pages.
purchases	courseSlug, courseTitle, amount, studentName, email, phone, paymentMethod, status, timestamps	Stores course purchase records and payment status.

5.2 Relationship Design

The purchase collection stores the selected course through courseSlug and courseTitle. This allows the purchase record to remain readable even if the static course list changes later. The video collection is independent and can be expanded for multiple playlists or course-specific mapping in future versions.

5.3 Security and Data Integrity

- User email is required and unique in the User schema.
- Passwords are hashed before saving through a Mongoose pre-save hook.
- Purchase payment method is restricted to upi, card, or netbanking.
- Purchase status is restricted to paid, pending, or failed.
- Required fields prevent incomplete user, video, and purchase documents.

6. Program Structure and Database Connections

The project structure separates backend entry point, database configuration, routes, models, templates, public assets, and seed data. This organization makes the codebase easier to understand, maintain, and deploy.

Path	Description
server.js	Main entry point. Configures Express, middleware, routes, static files, and starts the server.
db.js	Connects the application to MongoDB using Mongoose and MONGODB_URI.
routes/	Contains page, authentication, playlist, and purchase route files.
models/	Contains User, Video, and Purchase Mongoose schemas.
views/	Contains EJS templates rendered by the server.
public/	Contains CSS, HTML assets, images, thumbnails, and videos.
data/courses.js	Stores static course metadata and helper function for lookup by slug.
seedVideos.js	Seeds initial video records into MongoDB.

6.1 Database Connection

```
require("dotenv").config();
require("./db");
app.listen(PORT, "0.0.0.0", () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

The application reads configuration from environment variables. For local use, .env can contain PORT=8080 and MONGODB_URI=mongodb://127.0.0.1:27017/learnifyDB. For deployment, Render supplies PORT automatically and MongoDB Atlas can supply the production database connection string.

7. Code Implementation and GUI Construction

7.1 Backend Code Implementation

The backend implementation is based on Express routers. `server.js` registers common middleware, configures EJS, serves static files, connects routes, and attaches the error handler. The routes folder implements page navigation, login/register operations, playlist operations, and course-purchase logic.

```
app.use("/", pageRoutes);
app.use("/auth", authRoutes);
app.use("/playlist", playlistRoutes);
app.use("/buy-course", buyCourseRoutes);
app.use(express.static(path.join(__dirname, "public")));
```

7.2 Authentication Implementation

The User schema stores email and password. Before saving a new user, the password is hashed with `bcryptjs`. During login, the entered password is compared with the stored hash through a schema method. This improves security by avoiding plain-text password storage.

```
userSchema.pre("save", async function(next) {
  if (this.isModified("password")) {
    this.password = await bcrypt.hash(this.password, 10);
  }
  next();
});
```

7.3 GUI Construction

The graphical interface is built using EJS templates and static CSS. Important views include login, register, index/home, about, courses, playlist, buy-course, course-payment, payment-success, teachers, teacher-register, and contact pages. Course thumbnails and profile images are stored inside `public/images`.

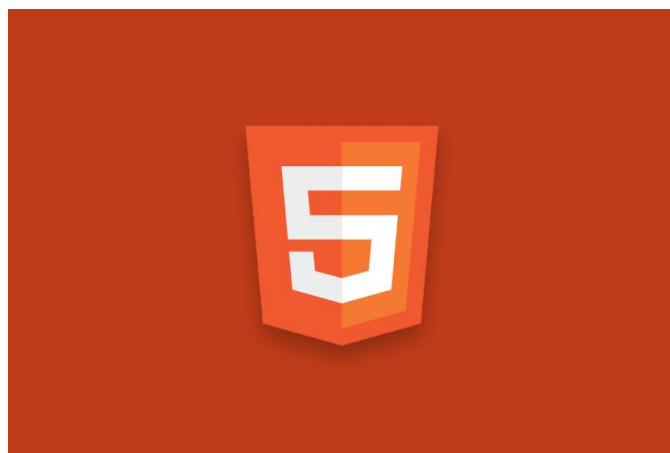


Figure: Course thumbnail asset used in Learnify.



Figure: HTML course visual asset used by the platform.

7.4 Course Purchase Flow

The buy-course route displays available courses, opens a course-specific payment page using the course slug, validates student and payment details, creates a Purchase document, and renders the payment-success view. This demonstrates complete form handling and database persistence.

8. System Testing

System testing verifies that all modules of Learnify work according to requirements. Testing focuses on route behavior, form validation, database operations, user authentication, static asset loading, EJS rendering, and deployment readiness.

8.1 Types of Testing Performed

Testing Type	Description	Expected Result
Unit Testing	Individual route handlers and model validations are checked.	Each module performs its specific function correctly.
Integration Testing	Routes are tested with database operations and EJS rendering.	Data moves correctly between forms, routes, models, and views.
Functional Testing	Login, register, courses, playlists, and payment flows are tested.	User receives correct page or validation message.
Database Testing	MongoDB records for users, videos, and purchases are verified.	Documents are inserted and retrieved correctly.
Security Testing	Password hashing and required-field validation are checked.	Passwords are stored as hashes and incomplete forms are rejected.
Compatibility Testing	Application pages are checked in modern browsers.	Pages render correctly and static assets load.

8.2 Sample Test Cases

Test Case	Input	Expected Output
Register new user	Unique email and password	User saved and login page shows success message.
Register duplicate user	Existing email	Register page shows duplicate-user error.
Login valid user	Correct email and password	Home page is rendered.
Login invalid user	Wrong email or password	Login page shows invalid credential error.
Open playlist	GET /playlist	Videos are fetched from MongoDB and playlist is rendered.
Buy course	Complete payment form	Purchase is saved and success page is rendered.

9. Conclusion

Learnify successfully demonstrates the design and implementation of a full-stack online learning platform using Node.js, Express.js, EJS, MongoDB, and Mongoose. The application provides structured course browsing, authentication, playlist management, teacher-related pages, and course-purchase recording.

The project helped in understanding server-side rendering, Express routing, middleware usage, MongoDB integration, Mongoose schema design, password hashing, static asset management, and cloud deployment preparation. Learnify can be extended further into a production-grade learning management system.

10. Future Scope

- Add JWT or session-based protected routes for authenticated-only course access.
- Create separate student, teacher, and admin dashboards.
- Map videos directly to courses and track lesson completion progress.
- Integrate a real payment gateway and invoice generation.
- Add course reviews, ratings, certificates, and quiz modules.
- Improve responsive design and accessibility across all pages.
- Add automated tests for routes, models, and validation logic.
- Deploy with MongoDB Atlas, monitoring, logging, and environment-specific configuration.

11. Bibliography and References

1. Node.js Official Documentation - <https://nodejs.org/en/docs>
2. Express.js Official Documentation - <https://expressjs.com/>
3. MongoDB Documentation - <https://www.mongodb.com/docs/>
4. Mongoose Documentation - <https://mongoosejs.com/docs/>
5. EJS Documentation - <https://ejs.co/>
6. bcrypt.js Package Documentation - <https://www.npmjs.com/package/bcryptjs>
7. Render Deployment Documentation - <https://render.com/docs>
8. Jon Duckett, HTML and CSS: Design and Build Websites, Wiley Publications.
9. Elmasri and Navathe, Fundamentals of Database Systems, Pearson Education.